

Cahier des Charges — Version Fusionnée

Open Source AI-Driven Embedded DevOps Platform (Zephyr RTOS + Yocto + Agents IA)

Équipe : Sahar, Ameni, Nour — 7 semaines restantes

1. Note d'orientation

Cette version enrichit le projet initial : l'écosystème **Arduino** est remplacé par **Zephyr RTOS** pour le microcontrôleur, et la passerelle (Raspberry Pi) utilise une image Linux sur-mesure construite avec **Yocto Project**. Ces deux ajouts sont des compétences très valorisées en entreprise, mais nettement plus difficiles qu'Arduino/Docker seul.

⚠ **Principe directeur accepté par l'équipe : l'apprentissage prime sur la perfection.**
On accepte la difficulté, mais on sécurise la démo finale avec un plan de secours (section 8).

2. Architecture détaillée (5 couches)

A. Frontend (React + TypeScript) : soumission de l'URL GitHub, suivi visuel du pipeline de build, dashboards Grafana intégrés (iframes), chatbot RAG.

B. Backend & Orchestration IA (FastAPI + Python) : pilote les 4 agents.

Agent	Rôle mis à jour
Agent 1 — Analyseur de code	Clone le dépôt (GitPython), détecte Zephyr RTOS (présence de <code>prj.conf</code> , <code>CMakeLists.txt</code>)
Agent 2 — Architecte infrastructure	Décide la stratégie de build, les variables d'environnement, la stratégie OTA
Agent 3 — Générateur d'artefacts	Génère (via templates Jinja2) les Dockerfiles et fichiers de configuration Zephyr/Yocto
Agent 4 — Assistant technique (RAG)	Base ChromaDB avec la doc Zephyr + logs d'erreurs, pour diagnostiquer les échecs de build

C. CI/CD & Build Runner : un conteneur Docker isolé qui installe `west` (l'outil de build de Zephyr), compile le firmware, exporte le binaire ou l'erreur.

 `west` : l'outil en ligne de commande officiel de Zephyr qui gère la compilation, les dépendances et le flashage du firmware.

D. Edge & Déploiement (Raspberry Pi) : image Linux minimale (Yocto) **ou Raspberry Pi OS standard en repli**, avec Docker Engine faisant tourner Mosquitto, InfluxDB, Grafana.

E. Flux de données : l'ESP32 (Zephyr RTOS, ou simulateur Python si le matériel est absent) publie les mesures capteur en MQTT vers la passerelle.

3. Stack technique complète

Domaine	Outils
Firmware	Zephyr RTOS (C), West CLI, ESP32
OS Gateway	Yocto Project (BitBake) + repli Raspberry Pi OS
Conteneurisation	Docker, Docker Compose
CI/CD	GitHub Actions, Build Runner Docker
Backend	Python, FastAPI
IA	API LLM (Claude/OpenAI/Groq/Ollama), ChromaDB, LangGraph (optionnel)
Frontend	Vite, React, TypeScript
IoT / Monitoring	MQTT (Mosquitto), InfluxDB, Grafana
Gestion de projet	Git, GitHub, GitHub Projects

4. Répartition des rôles

Sahar — IA & DevOps lourd (tes priorités conservées)

- **Docker complet** : infra backend, Mosquitto, InfluxDB, Grafana + Build Runner.
- **Agent 1** (détection Zephyr) + **Agent 4** (RAG avec doc Zephyr).
- Contribution à l'intégration de Docker dans l'image Yocto d'Ameni (support ponctuel, semaine 5-6).

Ameni — CI/CD & OS Embarqué

- **Yocto Project** : image Linux minimale pour Raspberry Pi (testée d'abord via QEMU, à distance).
- **Build Runner Docker** : conteneur qui compile un projet Zephyr via `west build`.
- **Agent 2** (décisions d'architecture, stratégie OTA).
- Tests automatisés + documentation.

Nour — Fullstack & IoT Embarqué

- **Firmware Zephyr RTOS** sur ESP32 (lecture capteur DHT22 + publication MQTT).
 - **Agent 3** (génération de fichiers via Jinja2).
 - **Frontend React/TypeScript** (formulaire GitHub, suivi de pipeline, chat RAG, iframes Grafana).
 - Intégration MQTT → InfluxDB → Grafana.
-

5. Contrats d'interface mis à jour (assemblage du travail)

■ **Contrat d'interface** : le format JSON exact échangé entre deux agents, figé dès le départ pour que chacune développe en parallèle sans attendre les autres, en utilisant un fichier `mock_*.json`.

Sortie de l'Agent 1 (Sahar → Ameni) :

```
json
{
  "framework": "Zephyr RTOS",
  "fichiers_detectes": ["prj.conf", "CMakeLists.txt"],
  "carte_cible": "esp32",
  "protocoles": ["MQTT", "WiFi"]
}
```

Sortie de l'Agent 2 (Ameni → Nour) :

```
json
{
  "strategie_build": "docker_west_build",
  "ota_active": false,
  "monitoring": true,
  "broker_mqtt": "mosquitto",
  "justification": "Projet Zephyr avec MQTT, build isolé nécessaire"
}
```

Liaisons non-IA (matrice d'interconnexion) :

- **Sahar ↔ Ameni** : Sahar fournit à Ameni la commande Docker exacte pour lancer le Build Runner ; Ameni lui renvoie le chemin du binaire généré ou le log d'erreur (utilisé par l'Agent 4 de Sahar).
- **Sahar ↔ Nour** : Sahar (Agent 1/backend FastAPI) fournit les endpoints REST à Nour, qui les consomme dans son frontend React.

- **Ameni ↔ Nour** : Ameni configure la passerelle Docker (MQTT/InfluxDB) ; Nour configure la carte pour publier vers l'IP de cette passerelle et intègre les graphiques Grafana en iframe.

6. Méthode de travail (rappel)

Chaque tâche part d'une branche Git dédiée, revue par Pull Request avant fusion. Point d'intégration hebdomadaire le vendredi : on remplace progressivement les mocks par les vrais composants. Test d'intégration global via `(docker-compose up)`. (Voir le Guide d'Intégration Équipe existant — la mécanique ne change pas, seuls les contrats JSON ci-dessus sont mis à jour.)

7. Planning sur 7 semaines (compressé depuis la version à 8 semaines)

Semaine	Sahar	Ameni	Nour
1	Docker de base + FastAPI + premier appel LLM	VM Linux + premiers pas Yocto (via QEMU)	Environnement Zephyr + projet "Blinky"
2	Infra Docker : Mosquitto	Poursuite image Yocto minimale	Zephyr + MQTT basique, maquette React
3	Infra Docker : InfluxDB + Grafana	Build Runner Docker (installation de <code>(west)</code>)	Vues React (formulaire GitHub, logs)
4	Agent 1 (détection Zephyr)	Agent 2 (stratégie + OTA)	Connexion MQTT → InfluxDB
5	Consolidation Docker + démarrage doc RAG	Build Runner complet (compile un vrai projet Zephyr) + CI/CD	Agent 3 (templates Jinja2)
6	Agent 4 (RAG) + support intégration Docker/Yocto	Optimisation image Yocto + support OTA basique	Firmware Zephyr complet sur ESP32 + dashboards Grafana
7	Intégration finale, tests, rapport	Intégration finale, tests, rapport	Intégration finale (UI + API), tests, rapport

8. Gestion des risques et plan de secours

Risque	Plan de secours
L'image Yocto n'est pas prête à temps	La démo finale utilise Raspberry Pi OS standard + Docker ; Yocto reste un chantier d'apprentissage documenté dans le rapport, basculé en production seulement s'il fonctionne à temps
Pas d'accès à un vrai Raspberry Pi	Ameni développe et teste son image Yocto via QEMU (émulateur), sans dépendre du matériel physique avant la fin
Zephyr RTOS trop complexe pour Nour en semaine 1-2	Partir des exemples officiels Zephyr (<code>samples/basic/blinky</code>), (<code>samples/net/mqtt_publisher</code>) et les adapter, plutôt qu'écrire le firmware from scratch
CI/CD (Ameni) prend du retard	Le Build Runner Docker peut être lancé manuellement en attendant que GitHub Actions soit finalisé — le pipeline automatique n'est pas bloquant pour la démo
Agent 4 (RAG, Sahar) trop dense en semaine 6	Ameni et Nour aident à rassembler les cas d'erreurs réels pour la base documentaire

9. Livrables finaux

1. Code source complet (dépôt GitHub).
2. Documentation technique (README, architecture, guide d'installation, plan de secours documenté).
3. Rapport de stage, incluant une section honnête sur les difficultés rencontrées avec Yocto/Zephyr (très valorisé par les encadrants — ça montre une vraie compréhension, pas juste un copier-coller réussi).
4. Démonstration de bout en bout (avec ou sans la vraie image Yocto, selon l'avancement réel).
5. Soutenance orale.

Ce document remplace le cahier des charges précédent. Il intègre la proposition d'Ameni (Zephyr + Yocto) tout en sécurisant la faisabilité en 7 semaines.